Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3780100

PDF Download
3780100.pdf
23 March 2026
Total Citations: 0
Total Downloads: 183

Published: 08 December 2025
Accepted: 23 November 2025
Revised: 14 October 2025
Received: 09 March 2025

Citation in BibTeX format

RESEARCH-ARTICLE

Efficient Timing Prediction and Optimization Using Derivable Gradient Boosting Machine Model at Placement Stage

XU HE, Hunan University, Changsha, Hunan, China

YUSHAN WANG, Hunan University, Changsha, Hunan, China

CHENJING YANG, Hunan University, Changsha, Hunan, China

RENJUN ZHAO, Hunan University, Changsha, Hunan, China

YIBO LIN, Peking University, Beijing, China

PEIYU LIAO, Chinese University of Hong Kong, Hong Kong, Hong Kong

View all

Open Access Support provided by:

Peking University

Chinese University of Hong Kong

Hunan University

National University of Defense Technology China

Efficient Timing Prediction and Optimization Using Derivable Gradient Boosting Machine Model at Placement Stage

XU HE, Information science and Engineering, Hunan University, Changsha, China
YUSHAN WANG, Hunan University, Changsha, China
CHENJING YANG, Hunan University, Changsha, China
RENJUN ZHAO, Hunan University, Changsha, China
YIBO LIN, School of Integrated Circuits, Peking University, Beijing, China
PEIYU LIAO, The Chinese University of Hong Kong, Hong Kong, Hong Kong
BEI YU, CSE, the Chinese University of Hong Kong, Hong Kong, Hong Kong
YAO WANG, Independent Researcher, Changsha, China
CHANG LIU, National University of Defense Technology, Changsha, China
YANG GUO, Computer, National University of Defense Technology, Changsha, China

Timing closure is a critical but effort-taking task in VLSI designs. In this paper, we investigate timing models and their associated optimization techniques suitable for integration into early design process. For accurate timing analysis at placement stage, an innovative timing prediction model that can be transformed into a derivable function is proposed, serving as a replacement for the conventional Elmore delay. One significant advantage of derivable timing models is their ability to be formulated as mathematical functions. This enables the derivation of gradients, which can be explored as direct timing optimization guidance during design process. Therefore, we further present a timing optimization method through model integration into iterative placement process. Experimental results demonstrate that our derivable timing model provides accurate prediction performance. Additionally, when integrated into the placement stage, it outperforms the use of Elmore delay and non-derivable models, achieving 12%~24% smaller Total Negative Slack (TNS) and Worst Negative Slack (WNS), with an average wirelength reduction of 1%~5%.

Additional Key Words and Phrases: Machine Learning, Static Timing Analysis, Timing Prediction, Timing Optimization, Placement

Xu He is the corresponding author (Email: dawn.hx@gmail.com).
Authors' Contact Information: Xu He, Information science and Engineering, Hunan University, Changsha, China; e-mail: dawn.hx@gmail.com; Yushan Wang, Hunan University, Changsha, China; e-mail: yushan_wang@hnu.edu.cn; Chenjing Yang, Hunan University, Changsha, China; e-mail: chenjing_yang@foxmail.com; Renjun Zhao, Hunan University, Changsha, China; e-mail: zhaorenjun@hnu.edu.cn; Yibo Lin, School of Integrated Circuits, Peking University, Beijing, China; e-mail: yibolin@pku.edu.cn; Peiyu Liao, The Chinese University of Hong Kong, Hong Kong, Hong Kong; e-mail: pyliao@cse.cuhk.edu.hk; Bei Yu, CSE, the Chinese University of Hong Kong, Hong Kong, Hong Kong; e-mail: byu@cse.cuhk.edu.hk; Yao Wang, Independent Researcher, Changsha, Hunan, China; e-mail: wangyaobsz@nudt.edu.cn; Chang Liu, National University of Defense Technology, Changsha, Hunan, China; e-mail: liuchang@nudt.edu.cn; Yang Guo, Computer, National University of Defense Technology, Changsha, Hunan, China; e-mail: guoyang@nudt.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-7309/2025/12-ART
https://doi.org/10.1145/3780100

Just Accepted

1 Introduction

Timing closure is a critical yet challenging task in VLSI design. Timing optimizations are performed throughout the entire design process. The earlier the intervention stage it involves in, the greater the room it has for timing optimization. Today, many timing optimization techniques are applied at early stages of design. For instance, during the placement stage, various methods have been proposed, i.e., net-based and path-based algorithms. However, aside from modifying the process algorithms for timing-driven optimization, it has also been observed that the accuracy of the timing model can significantly impact timing performance.

Accurate timing estimation before sign-off stage, i.e., at placement stage, is challenging due to the absence of essential connection physical information. As a result, traditional timing analysis methods rely on simplified circuit models, which reduce accuracy significantly as wire lengths and fan-outs grow. To address timing closure under inaccurate timing analysis, circuits are designed using a worst-case assumption, which adds an over-pessimistic timing margin to ensure the performance specification under process variations. Industrial experience has shown that, the immense gap between the performance predicted by EDA tools and the final silicon performance can be up to 30% [34]. In addition to the pessimistic timing margin, in early design stage, a large arbitrary clock uncertainty is inserted in order to cover up the net delay estimation "error" before routing. Consequently, the design might be optimized with a large mis-correlation with the sign-off net delay information, which could result in multiple design turn-around iterations and/or poor sign-off Power-Performance-Area (PPA) trade-offs.

To overcome the accuracy limitations of early timing analysis, recent methods have utilized machine learning (ML) models like random forest [6, 22], XGBoost [9], and Graph Neural Network (GNN) models [20]. These techniques have shown improved timing prediction accuracy.

However, a key challenge remains: integrating these accurate ML models into the design flow for practical performance evaluation. Most existing timing-driven algorithms rely on the Elmore model for timing analysis. Although the Elmore model is known to be inaccurate, it continues to be widely used in design tools for preliminary timing analysis due to its simple mathematical formula, which allows for computational efficiency and facilitates the derivation of timing optimizations in various timing-driven placement algorithms [7, 8, 11, 19, 28, 29, 32, 32, 33, 33, 35]. Therefore, to enable machine learning (ML) to provide more accurate timing optimization, ML models must not only improve their accuracy but also incorporate integration methods that can guide timing optimization effectively.

In this work, we aim to enhance both accurate timing analysis and corresponding optimization guidance method for timing model in placement stage, leading to more efficient circuit designs as shown in Fig. 1. Our focus is on developing a solution that combines the accuracy of ML-based model with the computational efficiency and derivability advantages of the Elmore model. Firstly, to match the benefits of the Elmore model, we propose a novel timing prediction model based on Gradient Boosting Machine (GBM). Similar to the Elmore model, our GBM model can be transformed into a derivable function. To enhance model accuracy, we generate a lookahead RC network during the placement stage for essential timing feature extraction. To further assess the effectiveness of our proposed timing model, we give the model integration method. As the model derivation can identify timing benefits related to connection criticality, the critical path can be optimized based on the partial derivative of our GBM model with respect to wirelength. In our implementation, the GBM model is integrated into the open-source placer DREAMPlace [30], employing net-weighting technology as a study case for timing optimization evaluation. Our experiments demonstrate that, compared to previous net-weighting methods, our model and its integration significantly reduce unnecessary wirelength adjustments during timing optimization, leading to superior Total Negative Slack (TNS) and Worst Negative Slack (WNS) results.

The main contributions of this paper can be summarized as follows:

Just Accepted

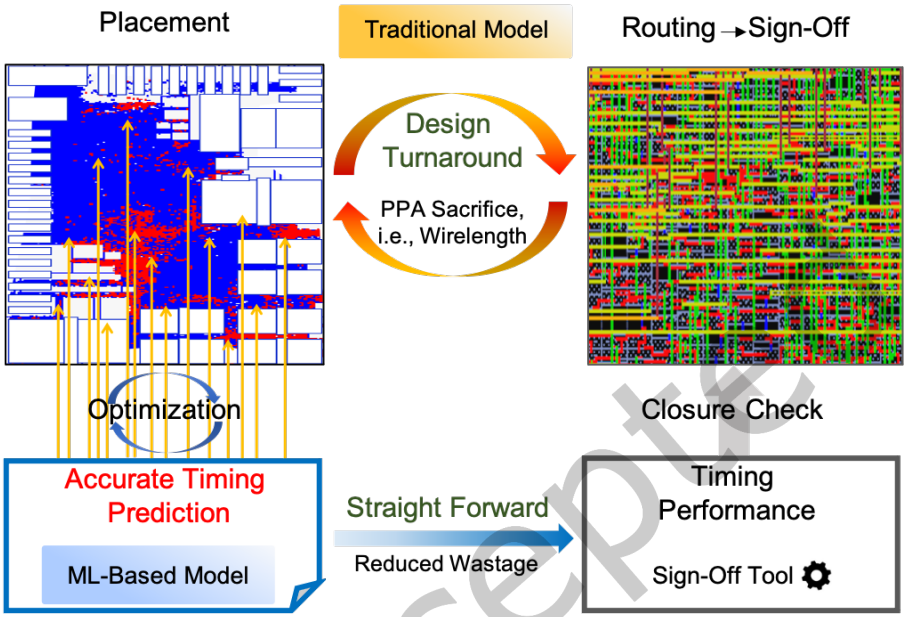


Fig. 1. Accurate timing analysis can reduce over-pessimistic design.

JUST Accepted

- We introduce a derivable function-based timing prediction model that can easily replace the traditional Elmore model. Significantly, its derivative provides direct and guiding influence on timing optimization, adding a crucial dimension to its applicability and effectiveness.
- We propose a novel feature extraction method based on look-ahead RC network, which compensates for the absence of routing information in early placement process.
- We propose a pin-to-pin weighting method that can directly identify critical pin-to-pin connections within the critical path. This approach minimizes resource wastage when optimizing the multi-pin network along the critical path.
- Experimental results have demonstrated that, under an industry advanced process technology (28nm), our machine learning-based delay model has achieved above 0.99 correlation with the sign-off timing on average. Moreover, the proposed timing model and integration approach in timing-driven placement can effectively improves timing performance while reducing wirelength loss caused by overly pessimistic predictions.

The remainder of this paper is organized as follows. Section 2 gives the motivation and previous related works on timing prediction and optimization techniques. Section 3 and Section 4 give the problem formulation and framework overview, respectively. In Section 5 and Section 6, we discuss the details of our timing model and integration method. Experimental results and conclusions are presented in Section 7 and Section 8, respectively.

2 Motivation & Related Works

STA is a critical component of the chip design flow, invoked repeatedly to evaluate timing performance and identify critical paths. Especially in the early design stages, both timing accuracy and optimization guidance are crucial aspects of the prediction model when integrating it into existing timing-driven workflows.

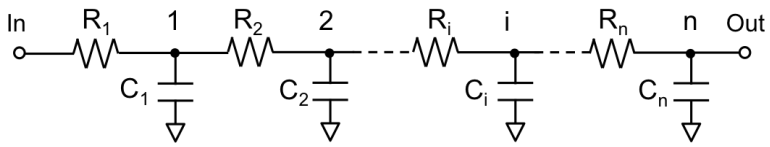


Fig. 2. Elmore delay model.

Currently, Elmore model [14], or the so-called first moment of the impulse response, remains the dominant choice for timing analysis during early design stages, such as placement process. This is due to its simple expression as a resistance-capacitance (RC) function, implicitly dependent on wirelength. Fig. 2 gives an RC chain example. Elmore delay can be considered as finding the delay through each segment i, as the segment resistance R_i times the downstream capacitance, and then taking the sum of the delays from the root to the sink. Segment number equals to the edge number of a path, e.g., n segments from input pin to output pin. Elmore delay of this RC chain is given in Equation (1). Given a routing result, each net can be modeled as an RC tree. For tree structure, the Elmore delay computation is similar, in which the downstream capacitance of node i is the sum of capacitance of all its branches.

$$Elmore(Out) = \sum_{i=1}^{n} R_i \times \sum_{j=i}^{n} C_j. \tag{1}$$

It is observed that the Elmore model simplifies delay calculation by expressing it directly as a function of resistance R and capacitance C, both of which have a linear relationship with wirelength. This allows timing-driven placement algorithms to translate the optimization of critical path delay into the optimization of wirelength. For example, net-weighting methods [8, 28, 29, 35] assign higher weights to timing-critical nets during placement. This prioritizes shorter wirelengths for these critical connections, ultimately reducing their delays and achieving timing optimization. Other techniques, like path-based techniques [7, 11, 19] directly model path timing constraints during placement, allowing for explicit optimization by manipulating cell coordinates.

While the Elmore model is highly integrable into the placement process, it suffers from inaccuracy compared to sign-off tool, particularly as wirelength increases or the number of fanouts in a net grows. Typically, wire delays are more challenging to calculate than gate delays due to their black-box nature. As shown in Fig. 3, statistical analysis reveals that as the timing path length increases, the contribution of interconnection wire delays becomes increasingly significant. Consequently, inaccuracies in delay estimation, especially for wire delays, cannot be ignored. For example, in some cases, compared to sign-off wire delay, the Elmore delay error can exceed 800ps, as illustrated in Fig. 3. Such inaccuracies can lead to incorrect critical-path timing evaluations and negatively affect timing optimization during placement.

In addition to the Elmore model, more accurate models have been proposed, such as the D2M (Delay with 2 Moments) delay metric [5], which accounts for the second moment of the impulse response, as well as other complex timing models like the Arnoldi model and the AWE (Asymptotic Waveform Evaluation) model, typically used at the post-routing stage. However, regardless of the method used, the issue of incomplete timing information before sign-off remains. This leads to discrepancies between the timing analysis conducted before sign-off and the final timing results after sign-off.

To narrow the gap between timing analysis and the sign-off golden result, machine learning methods can be used for timing prediction. Related work includes: delay prediction in the logic synthesis stage [31, 36, 37]; timing prediction in the placement stage [6, 20, 22]; wire delay prediction in the routing stage [9]; cross-tool prediction

Just Accepted

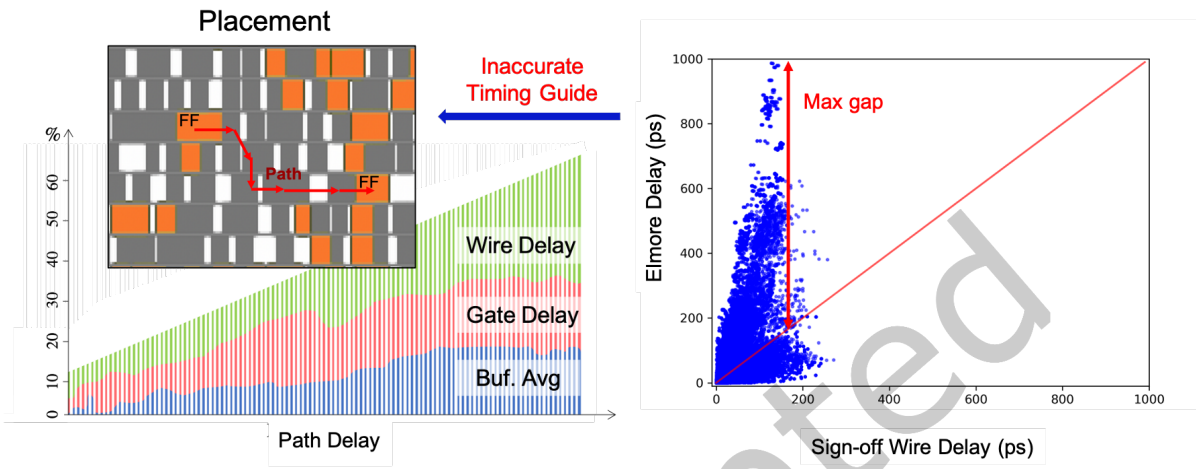


Fig. 3. The large gap between Elmore delay and sign-off wire delay may result in a significant inaccurate timing guide for critical path identification.

Just Accepted

in the sign-off stage [21, 24] , Signal Integration (SI) mode timing prediction [25] and Path-Based Analysis (PBA) timing prediction [18, 26], etc.

For timing analysis during the placement stage, prior prediction approaches encounter the following critical challenges: (1) **Lack of model-integration methodology**: The primary goal of timing analysis at this stage is to optimize timing during placement. However, the feasibility of replacing the traditional Elmore model with ML models needs further exploration, particularly regarding the integration method of the repeated model invocations during optimization. (2) **Trade-off between model accuracy and design pessimism**: The accuracy of timing models is affected by several factors, such as the selection of appropriate RC information features and the choice of ML algorithm. Over-simplified models may reduce prediction accuracy, resulting in pessimistic design risks. Conversely, more complex models often introduce significant computational overhead, resulting in substantial time costs when utilized frequently.

Both timing accuracy and optimization guidance are crucial aspects of the prediction model. Therefore, we aim to provide a timing prediction model that not only offers accurate predictions but also makes it easier to integrate into the placement optimization process. In this work, we evaluate our model on placement optimization guidance by integrating it into the net-weighting technique as a case study. It is important to note that the model derivability allows for its application to other timing optimization techniques, such as path-based methods.

3 Problem Formulation

In the problem formulation, besides timing accuracy evaluation, it is also crucial to incorporate timing considerations into the design flow. The term timing "slack" is commonly used to determine a critical timing path, which is defined as the difference of the required arrival time minus the arrival time. Negative slack means the path arrival time does not meet the requirement. The problem formulation of derivable timing model and integration for early timing optimization at placement stage can be described as follows:

Problem. Given netlists and input layouts, cell library, and timing reports generated by the golden sign-off timer, the objective is to train a derivable delay predictor that minimizes the discrepancy between the predicted delay

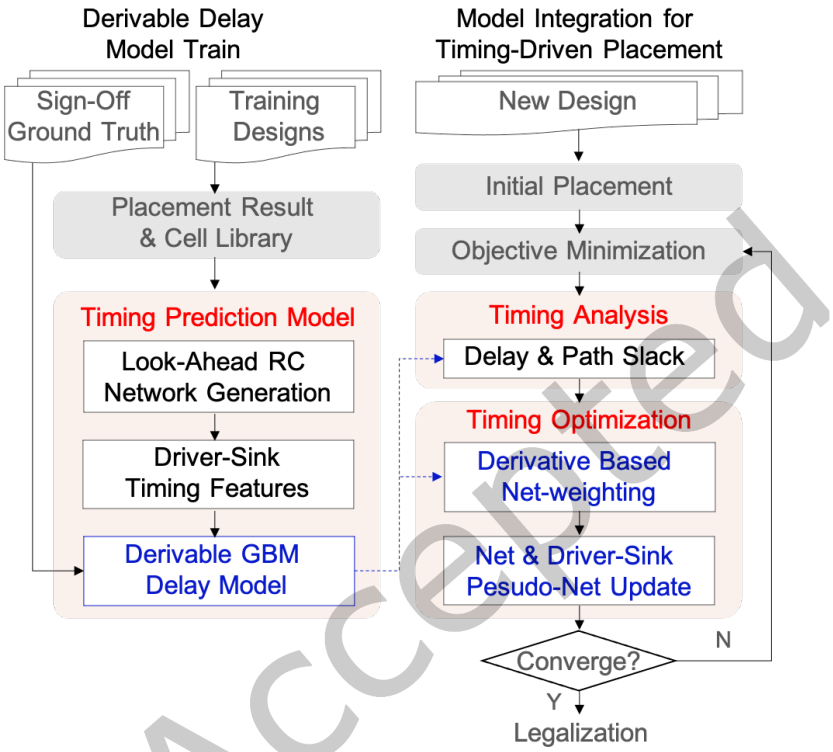


Fig. 4. Our timing analysis and optimization framework at placement stage.

Just Accepted

and the golden timing report. Moreover, for new netlist, the timing model can be effectively integrated into the placement process for timing optimization, i.e., total negative slack (TNS) and worst negative slack (WNS).

4 Methodology Overview

Fig. 4 provides an overview of our methodology. Our approach primarily consists of two parts: generating an accurate derivable timing prediction model, and integrating it into the placement optimization process to fully utilize its derivability. The modules related to timing, including model construction, analysis, and optimization, are shown in red. Relative to our earlier version in [22], the key differences, namely the derivable timing model and its corresponding optimization guidance, are highlighted in blue.

To generate derivable model, Gradient Boosting Machine (GBM) is utilized to predict the net delay. We transform this machine learning model into a derivable function form by creating derivable basic learners within GBM. This approach not only provides an efficient timing model, but also serves as a foundation for subsequent timing optimization during placement. In the model training process, given the placement input and technology libraries, we have to do the feature extraction for the net delay model. At the placement stage, the routing information is still unavailable. To improve the accuracy of timing prediction, we apply a fast routing analysis on the inputted placement layout to generate a look-ahead RC tree for each net. With the help of the generated RC network, the topology-related characteristics like wire-length, as well as the parasitic parameters like the resistance and capacitance of nets, can be extracted and fed to train the net delay model. The training dataset for each net

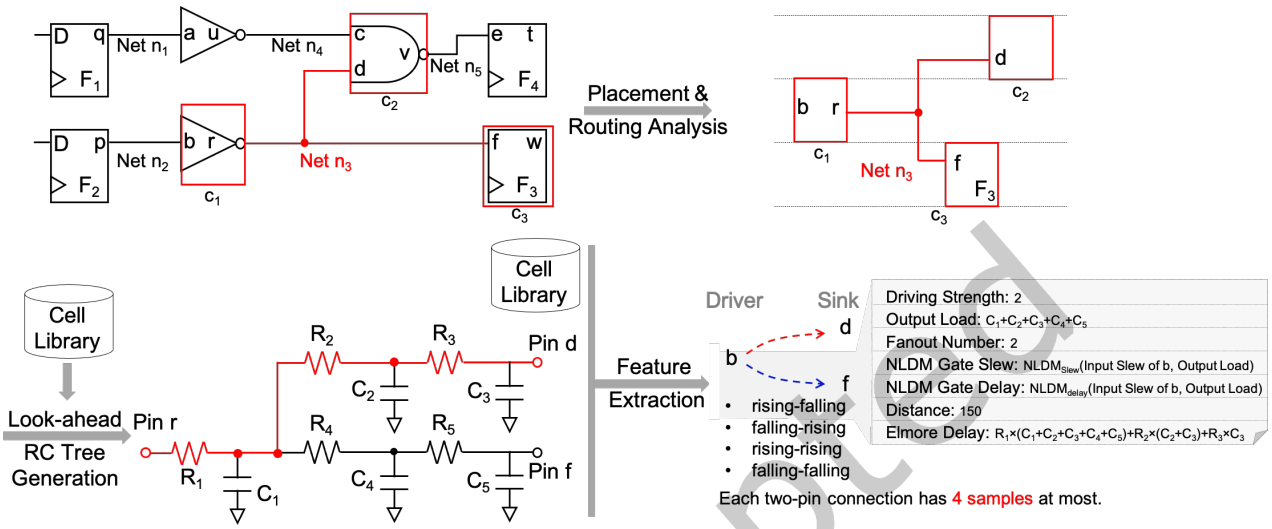


Fig. 5. Feature extraction example. For a multi-pin net n_3, it can be partitioned into two-pin nets from input pin b of driver c_1 to sinks d and f, respectively. Each two-pin net-based delay incorporates both driver gate delay and wire delay to the target sink. Based on the placement layout, a fast routing analysis is applied to generate a look-ahead RC tree of n_3. Finally, the timing features are extracted according to the generated RC tree.

Just Accepted

with timing label is extracted from a post-routing sign-off timing report using the PrimeTime, with the crosstalk considered and the timing analysis performed in signal integrity mode.

Essentially, since our model is derivable, to further validate the derivation effectiveness on timing optimization guidance, we adopt the net-weighting framework as a case study of model integration. During the model integration into the placement process, once the initial placement is provided, timing analysis is repeatedly invoked to guide timing optimization on the intermediate layout. For each net, given its timing features, the trained derivable model is utilized for the prediction of the net delay, followed by path timing computation. In the net-weighting approach, a proper net-weighting should assign higher weights on more timing-critical nets, with the hope that the placement engine will reduce the lengths of these critical nets and thus their delays to achieve better overall timing. Here, the interconnect length is derived using the aforementioned timing model, which allows for the calculation of timing derivation to determine the wirelength weight.

5 Derivable Timing Prediction

In this work, the objective of our timing analysis is to improve the accuracy of the traditional timing model, while keeping its characteristics of convenient usage to combine with or even replace traditional model in prior timing optimization algorithms. The key techniques include the generation of the look-ahead RC network, the feature selection & extraction method, and the derivable ML algorithm.

5.1 Look-ahead RC Network Generation

At the placement stage, the routing information is still unavailable. To improve the accuracy of timing prediction, we apply a fast routing analysis to generate a look-ahead RC network for the input placement layout. In such a way, not only the wirelength, but also the parasitic resistance and capacitance of the nets can be obtained for the correlation improvement of net delay with the post-routing results.

Table 1. Feature Summary in Our derivable Model

Type	Feature	Reference
Driver -Related	Driving Strength	Netlist
	Fanout Number	
	Output Load	Look-Ahead RC Network
	NLDM Gate Slew	Cell Libaray, Look-Ahead RC Network
	NLDM Gate Delay	
Interconnection -Related	Distance	Placement Result
	Elmore Delay	Look-Ahead RC Network
	Sink Slew	

To do fast routing analysis, we first partition each multiple-pin net into two-pin nets using Steiner tree algorithm [12]. Each two-pin net is then routed by L-shape routing. This routing analysis can be replaced by other simplified global routers as well for more sophisticated routing analysis.

5.2 Feature Extraction

In timing prediction, feature selection is of critical importance to the effectiveness of model application. The look-ahead RC network is applied for timing feature extraction. Table 1 lists the features for our net-based delay model.

Each net-based sample is a connection between a driver input pin and a target sink, i.e., $b \rightarrow d$ in Fig. 5, the features can be divided into driver-related features and interconnection-related features.

The driver-related features include: (1) Driving strength: It is extracted by the cell type, and it determines the output impedance of the driver. Typically, the higher the driving strength, the smaller the net delay. (2) Output load: It is the load capacitance of driver. (3) Fanout number: The number of sink pins of the net. (4) NLDM gate slew: It represents the speed of rise and fall transitions, and thus influences timing. (5) NLDM gate delay: It is the delay of the timing arc within the driving gate. It should be mentioned that, both gate slew and gate delay are not the golden result from the sign-off timer. Instead, they are obtained by the NLDM look-up table with two input indexes: the input slew and the output load of the driver, both of which are computed with the help of our look-ahead RC network.

The interconnection-related features include: (1) Distance: It is the Manhattan distance between the locations of the driving output pin and the target sink pin. (2) Elmore delay: We select the capacity and resistance from the RC tree, and compute the Elmore delay between the driving output pin and the target sink pin. (3) Sink slew: The total slew of all sink pins, along with its associated impulse and capacitive load, is calculated by the Elmore delay model. It represents the quality of the signal transmission.

Fig. 5 gives an example of feature extraction. Given a net n_3, we do routing analysis by partitioning n_3 into two-pin nets $b \rightarrow d$ and $b \rightarrow f$ using Steiner tree algorithm. Each two-pin net is then routed by L-shape routing. A look-ahead RC tree can be generated from the routing analysis and the cell library. The features of each two-pin net are then extracted. The extracted features of the two-pin net $b \rightarrow d$ are given in Fig. 5 as an example.

It should be mentioned that, to consider rising and falling delays in the same predictor, for each two-pin net, different data samples are generated according to the combinations of rising and falling signals, i.e., at most $2 \times 2 = 4$ combinations. For each two-pin net, its rising and falling samples are varied by different related-features and timing labels.

More timing features can further optimize timing prediction, but would require more feature computation and increase model complexity. To do the trade-off between timing performance and complexity, only the above timing features are selected in our placement.

Just Accepted

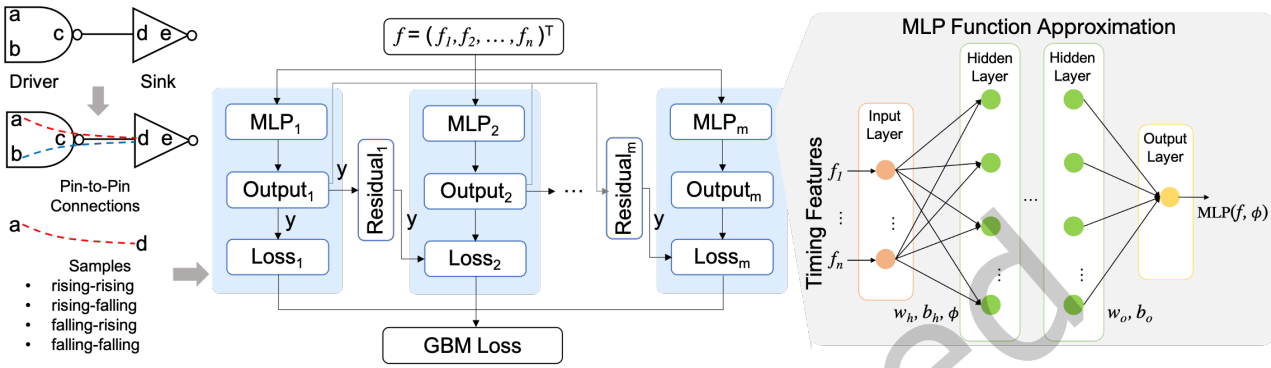


Fig. 6. Our derivable GBM driver-sink delay model with MLP as basic learner.

Just Accepted

5.3 Derivable GBM Model

We choose the GBM model, a kind of decision tree-based algorithm, for its proven effectiveness with tabular and discrete data [17], making it well-suited for pin-to-pin and net-based timing predictions, i.e. XGBoost, LightGBM. However, common GBM methods are not derivable, which, to some extent, constrains their usability and adaptability in the context of timing optimization during the placement process. To address this limitation, the soft gradient boosting machine (sGBM) framework [16] is applied. This framework ingeniously connects multiple derivable base learners into a cohesive, derivable system. It achieves this by introducing the local losses and the global losses, drawing inspiration from the GBM concept.

Our derivable basic learner makes use of the multilayer perceptron (MLP). MLP is a modern feedforward neural network composed of distinct layers, which includes at least an input layer, a hidden layer, and an output layer. At its core, MLP relies on three fundamental elements: weights, biases, and activation functions. The MLP model is essentially a function made up of multiple nonlinear and linear transformations. Importantly, each of these individual transformations is derivable, ensuring that the entire function expression is also derivable.

Specifically, assuming that a MLP contains an input layer, a hidden layer, and an output layer, the MLP function can be expressed as Equation (2).

$$MLP(f, \theta) = W_o \cdot \phi_{h_1}(W_{h_1} f + b_{h_1}) + b_o, \tag{2}$$

where $f = (f_1, f_2, ..., f_n)^T$ represents the input feature vector, $\theta = \{W_{h_1}, b_{h_1}, W_o, b_o\}$ represents the set of model parameters. W_{h_1} and b_{h_1} are the $n \times n$ weight matrix and $n \times 1$ bias vector of the hidden layer, and $\phi_{h_1}(\cdot)$ is the activation function. The W_o and b_o are the $1 \times n$ weight matrix and bias value of the output layer.

Similarly, the function expression of an MLP with k hidden layers can be denoted as Equation (3).

$$\begin{aligned} MLP(f, \theta) =& W_o \cdot \phi_{h_k}(W_{h_k}(\ldots \phi_{h_1}(W_{h_1} f + b_{h_1}) \ldots) \\ &+ b_{h_k}) + b_o, \end{aligned} \tag{3}$$

In our GBM model, its corresponding function can be expressed as the sum of the outputs of all basic MLP leaners. The function expression of our GBM model is given in Equation (4).

$$\begin{aligned} DelayGBM(f) =& lr \cdot \sum_{i=1}^{n} MLP_i(f, \theta_i), \\ Softplus(z) =& \ln(1 + \mathrm{e}^z), \end{aligned} \tag{4}$$

where n is the number of basic MLP learner, and lr is the learning rate, which controls the contribution of each base learner to the overall model. The choice of activation function is very important to the MLP, and various activation functions are examined in our experiments, including the commonly used $Sigmoid$, $Tanh$, and $Relu$ functions. However, both $Sigmoid$ and $Tanh$ functions suffer the problem of gradient vanishing, and the $Relu$ function is not derivable at all points. To solved the problems, we have turned to the $Softplus$ function, which is a smoothed version of $Relu$ function. The derivative of the $Softplus$ function is the $Sigmoid$ function, for which does not suffer from the gradient vanishing issue fortunately, thereby resolves all the aforementioned problems.

6 Timing Optimization with Derivable Model

In addition to model accuracy, we also investigate the corresponding timing optimization method by utilizing the derivability of the model. In this study, we adopt the net-weighting technique to demonstrate timing optimization using derivable timing model, as net-weighting technique is commonly employed in timing-driven placement and also applicable to other non-derivable model for performance comparison.

In net-weighting technique, it assigns different weights to different nets such that the placer minimizes the weighted sum of net wirelength shown in Equation (5).

$$\min_{x,y} \sum_{e \in E} w(e) \cdot WL(e; x, y) + \lambda D(x, y), \tag{5}$$

where x, y are cell coordinates, $w(e)$ is the weight of net e, and the density constraint is usually transformed into a density penalty term with the density penalty weight λ. In an analytical placement, this optimization problem is solved by gradient descent on the objective function.

Although the net-weighting technique appears to be straightforward, generating an effective net-weighting scheme is still not an easy task. Assigning bigger weights to critical nets typically reduces their wirelength and delays, but it can also lead to length growth and worsen timing of other near-critical nets, therefore the emergence of new critical path. To solve the problem, we introduce a dynamic net-weighting method, which involves updating weights based on the above derivable timing model during placement process.

Many dynamic net-weighting techniques primarily set weights for the whole nets $e \in E$ as shown in Equation (5). For those methods, if only part of the driver-sink interconnections of multi-pin net e belong to the critical paths with negative slacks, the net weight $w(e)$ is updated according to the worst slack. However, when dealing with a multi-pin net e that involves multiple driver-sink connections, it becomes necessary to determine whether these connections is truly a critical path or not, and then should be optimized explicitly if it is. For this reason, it's crucial to assign different weights to the lengths of driver-sink connections from different paths to accurately reflect their importance in the overall timing.

In our implementation, we add two-pin pseudo-nets to the driver-sink connections for whose worst path slacks are below zero. For a two-pin pseudo-net e, its net weight computation is given in Equation (6), in which the belonging path slack and delay of the pseudo-net e are considered.

$$w(e) = \frac{dWL(e)}{dMax} \times wireRatio(e), \tag{6}$$

where $wireRatio(e)$ is the wire and gate delay ratio of driver-sink pseudo-net e, in which the gate delay is obtained from the cell library. The wire delay can be calculated by subtracting the gate delay from the predicted driver-sink net-based delay. Alternatively, the ML model can be directly adjusted to predict the wire delay specifically for the

Just Accepted

timing arc from the driver output to the sink input. The $dWL(e)$ is applied to account for the impact of timing delay change to wirelength $WL(e)$ by partial derivative on $DelayGBM(f)$, and $dMax$ is the maximum derivative of all pseudo-nets.

For a MLP learner in GBM model with one hidden layer as an example, using the timing delay model $DelayGBM(f)$ with features f of e in Equation (4), the wirelength impact can be estimated in Equation (7). For each MLP learner in GBM model with multiple hidden layers, it can be derived similarly.

$$\begin{aligned} dWL(e) =& \frac{\partial DelayGBM(f)}{\partial WL(e)} \\ =& W_o \cdot \frac{\partial Softplus(W_{h_1} f + b_{h_1})}{\partial WL(e)} \\ =& W_o W_{h_1} \cdot \frac{1}{1 + \mathrm{e}^{-(W_{h_1} f + b_{h_1})}} \cdot \frac{\partial f}{\partial WL(e)} \end{aligned} \tag{7}$$

Referring to the feature vector f, wirelength mainly involves the Elmore delay and distance. For feature distance of e, its partial derivative with respect to $WL(e)$ is 1. For feature Elmore delay $Elmore(e)$, the partial derivative is given in Equation (8) [35].

$$\frac{\partial Elmore(e)}{\partial WL(e)} = rc \cdot WL(e) + cR_d, +rC_l, \tag{8}$$

where r and c are the unit length wire resistance and capacitance, respectively, R_d is the output resistance of the driver, and C_l is the load capacitance.

It implies that given a specific technology setting with trained GBM model, large weights on wirelength-related features, long wire with weak driver and large load will exhibit higher sensitivity to the same wirelength change.

For other features, i.e., gate slew, since they are tabular or discrete data and cannot be derived by wirelength, their partial derivative with respect to $WL(e)$ is simply set to 0. Above net-weighting of driver-sink connections can be extended to handle more general timing model by similar computation of deriving features with respect to $WL(e)$.

7 Experimental Result

The proposed placer approach is implemented in C++17 and timing model trained with Python 3.8, and then evaluated on a workstation with an Intel Xeon Core (@3.20 GHz) and 256 GB DDR4 memory. The experiment setups and result analysis are illustrated and discussed in detail as follows.

7.1 Dataset Generation

To evaluate the generalization ability of timing prediction and optimization, we implement existing open-source RTL designs to generate golden timing samples through the complete RTL-to-GDSII flow. This includes ITC'99 benchmark circuits [13] and open-source RISC-V circuits such as `CV32` [1], `CVA6` [2], and XuanTie Open `C910` [3], all synthesized and implemented using a 28nm industry process. All circuit designs are first synthesized using Synopsys Design Compiler. Physical design (P&R) is then conducted using Cadence Innovus, followed by parasitic extraction with Synopsys StarRC. Finally, Synopsys PrimeTime is executed in signal integrity mode to generate timing reports as the ground truth.

Among the open-source RISC-V CPUs, `CV32` is a 32-bit RISC-V core with a 4-stage pipeline, `CVA6` is a 64-bit Linux-capable 6-stage RISC-V core with a 32KB L1 data cache and a 16KB L1 instruction cache, and `C910` is a high-performance 64-bit multi-core processor featuring 64KB L1 instruction and data caches along with a 1MB L2 cache. These RISC-V CPUs differ significantly from the ITC'99 circuits, exhibiting greater complexity and

Just Accepted

Table 2. Benchmark Suite Information

Type	Circuit	#Cells	#Nets	#FFs	#Sample
ITC'99	b11	3,000	3,120	132	16,724
	b12	5,644	6,100	476	33,802
	b14	41,304	42,108	1,024	182,848
	b17	104,172	110,124	6,084	649,326
	b22	92,800	95,980	3,120	529,560
RISCV	CV32	120,537	129,804	8,055	903,062
	CVA6	194,702	197,240	23,195	1693,632
	C910	2,310,629	2,338,764	313,806	20,532,192

The circuit data is the total number of all implementations.

larger sizes. By utilizing these datasets, we aim to assess the effectiveness of timing prediction and optimization in handling more challenging and realistic designs.

Since each implementation of an ITC'99 circuit is small in scale, it is not ideal for evaluating prediction models because prediction accuracy can fluctuate significantly. For the same reason, CV32, although slightly larger than the largest ITC'99 case, remains a relatively simple RISC-V circuit. To obtain sufficient net samples, we variate several tool parameters and generate 4 different design implementations for ITC'99 circuits and 3 different implementations for circuit CV32. This dataset enrichment approach has also been adopted in prior work [6]. In contrast, CVA6 and C910 are much larger than ITC'99 and CV32. Their larger size not only produces a greater number of net samples but also provides substantial diversity in sample characteristics, such as wirelength and fanout range, which is sufficient for evaluating prediction performance. Therefore, for these large-scale circuits, we did not generate additional implementations.

A summary of the circuit characteristics is provided in Table 2, including the total number of samples across all implementations, where each sample refers to a pin-to-pin connection. Half of the samples from circuits CV32, CVA6 and C910 are randomly selected for training, while the remaining samples are used for testing. This includes the entire dataset of ITC'99 for cross-validation. There is no intersection between training and testing data.

It should be mentioned that, in the dataset generation process from RTL to sign-off, netlist may change across different stages. It is impractical to track the netlist changes from placement-stage to sign-off stage to produce dataset, as only a very small number of nets remain identical between these two stages. Instead, we extract physics-related information from the sign-off netlist that is equivalent in scope to the information obtainable at the placement stage. A dataset is then constructed from the sign-off netlist along with the corresponding sign-off timing labels. To ensure consistency with the features available at the placement stage, all post-placement details in the sign-off netlist are excluded.

7.2 Timing Prediction Validation

7.2.1 Delay Prediction Analysis. To demonstrate the effectiveness of our "DelayGBM" model, we conduct a comparative analysis against the "Elmore" delay model and prior timing prediction models, i.e., XGBoost-based model "MLCAD'22" [10] and our earlier work "DAC'22" [22] based on Random Forest (RF). It is important to note that all previous models were developed using their proposed features and methodologies within our timing predication framework, where rising and falling arcs are considered separately. Additionally, since the "DAC'22" work uses a net-based approach, we ensure fairness by adding the driving gate delay to both the Elmore delay and the wire delay [10].

The comparison on the net-based delay is given in Table 3. The columns "Mean" (ps), "Max" (ps), and "Cor" are the mean error, maximum error, and correlation with respect to the sign-off timing from PrimeTime, respectively.

Just Accepted

Table 3. Timing Prediction Comparison on Original Implementation

	Elmore			MLCAD'22				DAC'22				DelayGBM			
Circuit	Cor	Max	Mean	Cor	Max	Mean	CPU	Cor	Max	Mean	CPU	Cor	Max	Mean	CPU
b11	0.98	70.76	1.82	0.98	69.77	2.13	0.01	0.98	70.93	2.54	0.01	0.99	70.52	1.39	0.00
b12	0.99	28.40	2.23	0.99	36.48	2.83	0.02	0.99	25.65	2.87	0.02	0.99	27.90	1.87	0.00
b14	0.98	100.78	4.90	0.98	86.86	5.48	0.06	0.99	57.56	4.19	0.07	0.99	49.27	2.97	0.02
b17	0.98	85.58	4.49	0.98	82.95	5.17	0.22	0.98	88.99	4.34	0.29	0.99	84.83	3.38	0.04
b22	0.99	98.52	4.11	0.99	104.23	4.85	0.19	0.99	74.39	3.99	0.22	0.99	70.63	2.77	0.03
CV32	0.95	186.84	4.91	0.97	118.25	4.42	0.11	0.98	101.06	3.38	0.13	0.98	56.21	2.11	0.03
CVA6	0.85	1123.95	11.76	0.95	330.70	9.18	0.06	0.97	177.40	5.65	0.06	0.98	200.51	4.40	0.01
C910	0.89	1536.53	11.69	0.96	648.34	8.43	0.55	0.96	320.39	6.72	0.78	0.98	337.65	5.05	0.11
Avg.	0.95	403.92	5.74	0.98	184.70	5.31	0.15	0.98	114.55	4.21	0.20	0.99	112.19	2.99	0.03

Table 4. More Challenge Timing Prediction Comparison with Wirelength Enlargement

	Elmore			MLCAD'22				DAC'22				DelayGBM			
Circuit	Cor	Max	Mean	Cor	Max	Mean	CPU	Cor	Max	Mean	CPU	Cor	Max	Mean	CPU
b11	0.76	187.47	14.69	0.81	167.91	14.55	0.00	0.91	100.88	7.60	0.01	0.92	86.68	6.12	0.00
b12	0.80	167.74	17.99	0.84	171.53	18.29	0.00	0.92	125.70	10.38	0.01	0.93	108.69	8.56	0.00
b14	0.81	209.52	22.37	0.89	163.37	22.18	0.03	0.94	121.41	9.40	0.04	0.95	111.77	7.03	0.00
b17	0.85	232.94	14.81	0.92	123.70	13.72	0.05	0.95	125.32	6.52	0.13	0.95	120.90	5.89	0.01
b22	0.84	190.47	13.01	0.90	155.88	12.08	0.04	0.95	128.70	5.22	0.06	0.96	126.56	4.66	0.01
CV32	0.89	190.80	8.24	0.93	127.13	7.26	0.03	0.97	97.02	3.39	0.09	0.97	91.40	3.06	0.01
CVA6	0.85	1123.95	11.76	0.95	337.94	9.30	0.06	0.98	135.09	4.82	0.12	0.98	155.92	4.49	0.01
C910	0.89	1536.53	11.69	0.96	649.15	8.50	0.64	0.98	256.57	5.36	1.39	0.98	334.05	5.10	0.08
Avg.	0.84	479.93	14.32	0.90	237.07	13.23	0.11	0.95	136.33	6.59	0.23	0.96	141.99	5.61	0.02

The column "CPU" presents the inference run-time in seconds. In our implementation, the look-ahead RC network generation takes up about 14% of total inference run-time averagely, in which the Steiner tree construction accounts for more than half of it. Normally, the larger a net is, the longer run-time it takes. The training time for "DelayGBM" is 1543 seconds for the total 11.5M training data, respectively.

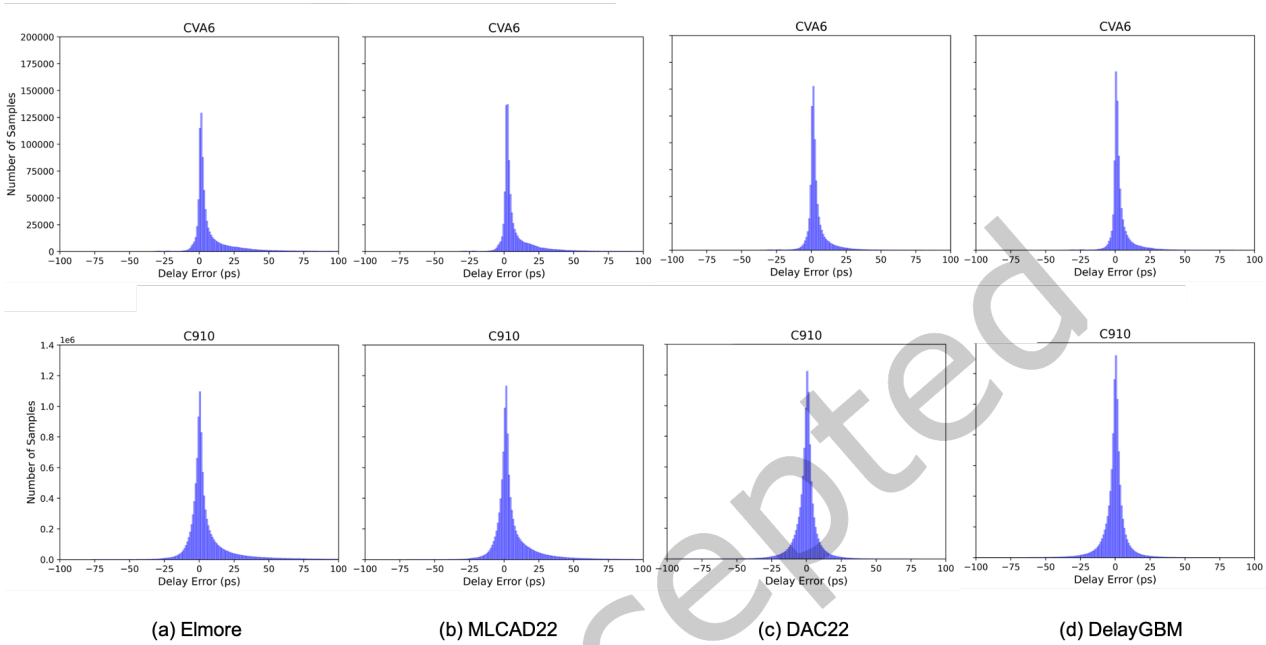
As shown in Table 3, all methods achieve a "Cor" greater than 0.95, making it difficult to differentiate their performance. This is due to the small circuit scale of ITC'99, which results in short wirelength and a wire delay proportion of less than 5% relative to gate delay. Since wire delay analysis is generally considered more challenging than gate delay analysis, we intentionally increase the circuit chip size and reduce cell density to lengthen cell distances and thereby increase the proportion of wire delay. In the following performance analysis, we use dataset derived from physical implementations with sparse cell distribution, thereby presenting a more challenging scenario characterized by increased wire delay.

In Table 4, as the wire delay proportion increases, the accuracy of the Elmore delay decreases significantly, while the "Cor" metric of ML-based models decreases by less than 8%. Among these models, the average "Cor" of "DelayGBM" remains above 0.95. This high consistency with the sign-off delay effectively avoids the pessimistic estimations seen with the Elmore delay. It is also worth noting that "DelayGBM" demonstrates a prediction performance equivalent to or even better than previous non-derivable models, indicating that a derivable learning model can be a more suitable substitute for non-derivable models in timing prediction.

Error distributions of the delay predictions are depicted in Fig. 7 for the cross validation results on CVA6 and C910. Results on other circuits and designs are similar. It is shown that our prediction errors are generally concentrated near ±0 with small spread, which indicates the effectiveness of our model.

We also analyzed other related works on timing prediction. For instance, the results of the work [9] is very similar to those of "MLCAD'22", with a correlation difference of less than 0.01, and thus are not listed due to table space constraints. In work [6], the timing analysis is overly simplified, such as assuming the maximum delay for driver with multiple input pins and not differentiating between rising and falling delays, instead using

Just Accepted



(a) Elmore (b) MLCAD22 (c) DAC22 (d) DelayGBM

Fig. 7. Comparison of error distribution of net delay prediction on circuit CVA6 and C910.

Just Accepted

the worst-case scenario instead. These simplifications lead to significant pessimism in the predictions, resulting in low sign-off correlation below 0.8.

To assess the effectiveness of the selected features for timing prediction, we extend the feature set beyond those listed in Table 1 by incorporating several commonly used timing-related features: (1) Segment-Num: the number of edges from the output to the input in the RC network; (2) Context-Elmore: for a given target sink, the other sinks are defined as context sinks, and the sum of Elmore delays between the driving pin and these context sinks is referred to as Context-Elmore; (3) D2M: the second moment of the impulse response; (4) Sink-Cap: the total input capacitance of all sinks in a net; (5) RUDY: the Rectangular Uniform wire Density of a net, serving as a routing congestion indicator; and (6) HPWL: the half-perimeter of the bounding box of the RC network. Experimental results indicate that the overall correlation remains nearly unchanged compared to the baseline feature set, with variations limited to within ±0.003 across different cases. However, incorporating these additional features into the full softGBM set introduces extra computational overhead, increasing inference runtime by approximately 30%.

7.2.2 Path Timing Analysis. We integrate the wire delay prediction results into OpenTimer [23] for path timing analysis. By traversing the path in topology order level-by-level, the endpoint arrival time can be obtained. The overall predictions of the endpoint arrival time are summarized in Table 5. Compared with other methods, our slack errors are the lowest, and our average correlation is about 0.94.

As shown in Fig. 8 for CVA6 and C910, compared to the Elmore and "MLCAD'22" methods, the distributions of our arrival path predictions closely align with the PrimeTime results, as indicated by the solid line, demonstrating a better match with the golden results from PrimeTime.

Table 5. Comparison on Endpoint Arrival Time

	Elmore			MLCAD'22			DAC'22			DelayGBM		
Circuit	Cor	Max	Mean	Cor	Max	Mean	Cor	Max	Mean	Cor	Max	Mean
b11	0.80	352.51	186.08	0.82	292.53	176.21	0.88	144.70	82.65	0.90	112.26	62.64
b12	0.70	339.06	180.59	0.78	330.20	182.55	0.77	175.13	91.86	0.78	142.21	70.67
b14	0.96	982.43	694.01	0.95	825.03	600.70	0.96	382.80	209.34	0.96	302.05	139.35
b17	0.93	869.47	438.19	0.95	624.33	354.29	0.97	199.20	71.75	0.97	157.27	36.90
b22	0.94	850.11	499.52	0.96	667.00	399.69	0.98	234.96	87.54	0.98	176.03	42.76
CV32	0.99	626.37	343.14	1.00	488.77	278.69	1.00	150.65	56.36	1.00	108.66	23.64
CVA6	0.93	1180.83	374.18	0.97	787.47	190.87	0.99	660.13	73.79	0.99	691.58	54.00
C910	0.89	1896.44	234.40	0.96	1126.89	155.84	0.98	560.51	45.19	0.98	529.32	39.32
Avg.	0.89	887.15	368.77	0.92	642.78	292.36	0.94	313.51	89.81	0.95	277.42	58.66

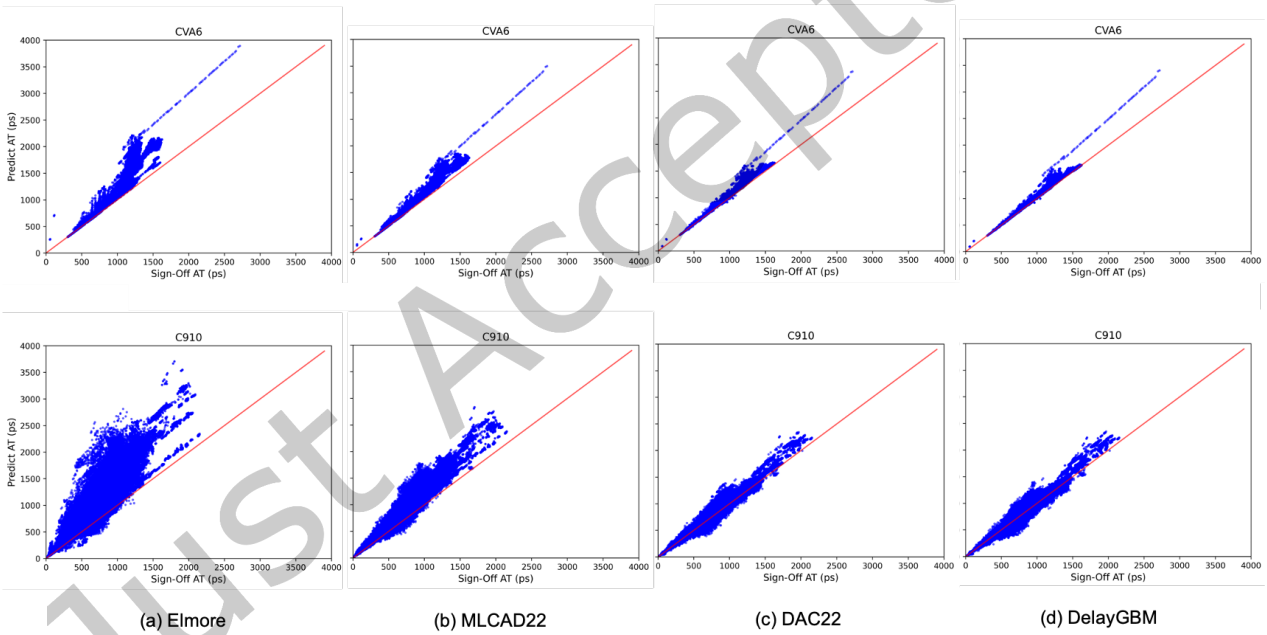


Fig. 8. Analysis of endpoint arrival time on circuit CVA6 and C910.

Just Accepted

A key application of timing analysis is the identification of timing-critical paths with negative endpoint slacks. Path slack is defined as the difference between the required time and the arrival time at an endpoint, such as a flip-flop (FF) or a primary output. The slack value is crucial to identify violated paths that require further timing optimization. In our experiments, slacks greater than a threshold, i.e., -15ps in PrimeTime report, are classified as non-critical paths, while negative slacks below this threshold are classified as critical paths. This is because minor negative violations can be easily corrected through cell library retiming, and the margins provided by on-chip variation (OCV) and clock uncertainty settings during verification ensure that such small violations do not result in functional failures.

Table 6. Critical Path Classification

Cir	PrimeTime		Elmore				MLCAD'22				DAC'22				DelayGBM			
cuit	TNS	WNS	TNS	WNS	TPR	TNR	TNS	WNS	TPR	TNR	TNS	WNS	TPR	TNR	TNS	WNS	TPR	TNR
b11	-0.4	-36	-7	-275	100	16	-6	-237	100	14	-1	-84	100	64	-1	-55	100	79
b12	0	0	-17	-225	/	16	-17	-233	/	17	-1	-71	/	73	-0.2	-42	/	95
b14	0	0	-205	-767	/	15	-160	-596	/	16	-4	-108	/	77	-0.2	-48	/	99
b17	0	-13	-661	-740	/	22	-427	-487	/	28	-1	-71	/	99	0	-11	/	100
b22	0	-8	-465	-712	/	24	-324	-528	/	27	-2	-71	/	93	0	-12	/	100
CV32	-632	-305	-2125	-899	100	35	-1804	-767	100	37	-831	-401	100	96	-712	-347	100	99
CVA6	-3024	-1492	-17136	-2669	100	27	-9113	-2276	100	29	-4676	-2147	100	55	-4067	-2180	100	62
C910	-3968	-1256	-69041	-2522	100	60	-35211	-1751	100	68	-6930	-1315	99	93	-5239	-1347	99	95
Avg.	-953	-389	-11207	-1101	100	27	-5883	-859	100	29	-1556	-533	100	81	-1252	-505	100	91

The results of the critical paths classification are summarized in Table 6. The column "TNS" (ns) and "WNS" (ps) denotes the total negative slack and the worst negative slack. The column "TPR" (%) and "TNR" (%) are the true positive and negative ratio, which represent the ratio of the number of truly critical and non-critical paths correctly identified by a classifier versus the total number of critical and non-critical paths reported from PrimeTime, respectively. A perfect TPR or TNR is 1.

As shown in Table 6, based on our accurate prediction of net delay, our "TNS" is closer to the golden result of PrimeTime. Furthermore, while achieving a "TPR" of 100%, our method has a much lower false alarm, resulting in more than 60% higher "TNR" compared to Elmore and "MLCAD'22" methods, which means our method can be effectively used as a timing estimator for timing optimization during the placement stage.

7.3 Timing-Driven Placement Performance Analysis

During placement optimization, we employ the acquired model "DelayGBM" instead of the Elmore delay for timing evaluation. Utilizing the timing results from "DelayGBM", pseudo-nets are added with timing slack less than 0 during the placement process, and the net weights are adjusted accordingly. As our timing optimization methodology is integrated into the open-source tool DREAMPlace, we conduct a comparison with both the open-source DREAMPlace [30] and net-weighting-based timing driven DREAMPlace 4.0 [29] with Elmore delay analysis. In performance evaluation of the ML-based model, "DAC'22" and "DelayGBM" demonstrate comparable accuracy in timing prediction. To assess the effectiveness of the derivable "DelayGBM" model in time-driven optimization, we also integrate "DAC'22" with RF model as a non-derivable model in placement for a comparative experiment. It should be noted that the integration method for the non-derivable model "DAC'22" involves simply replacing the Elmore model in DREAMPlace 4.0, without the addition of pseudo-nets for driver-sink connection optimization on critical paths.

Due to the small scale of ITC'99 circuits and accordingly too short wirelength, optimizing timing by reducing wirelength is not practical. Therefore, our primary focus is on the timing-driven optimization for larger RISC-V circuits, i.e., CV32, CVA6 and C910, as shown in Table 7. The "TNS" and "WNS" is consistent with the metric applied in [29]. Our approach "DelayGBM" achieves an obvious improvement compared to the basic version "DREAMPlace". On average, it reduces TNS and WNS by 34% and 29%, respectively.

Compared to "DREAMPlace 4.0", "DelayGBM" achieves a 1% reduction in HPWL while improving TNS and WNS by 24% and 14%, respectively. This improvement can be attributed to the integration of a derivable model and its corresponding derivatives into the timing optimization process, which effectively reduces wirelength performance loss while incurring significantly less runtime overhead compared to the integration of the non-derivable model in "MLCAD'22" and "DAC'22". Furthermore, we can further employ timing derivatives directly on driver-sink connections of critical paths. This approach results in a more significant improvement in both TNS and WNS.

Just Accepted

Table 7. Performance on Timing-Driven Placement Analysis

Circuit	Method	HPWL	TNS	WNS	CPU
CV32	DREAMPlace	4.70E+06	-1094	-702	20
	DREAMPlace4.0	5.43E+06	-983	-608	49
	MLCAD'22	4.67E+05	-977	-620	369
	DAC'22	5.03E+06	-978	-621	374
	DelayGBM	4.89E+06	-949	-586	40
CVA6	DREAMPlace	2.91E+07	-33837	-2593	43
	DREAMPlace4.0	3.51E+07	-29589	-2206	241
	MLCAD'22	3.14E+07	-28495	-2342	901
	DAC'22	3.13E+07	-27967	-2334	916
	DelayGBM	3.05E+07	-27551	-2224	246
C910	DREAMPlace	4.95E+08	-315507	-5742	2229
	DREAMPlace4.0	5.11E+08	-293036	-5138	2809
	MLCAD'22	5.22E+08	-299016	-4898	5896
	DAC'22	5.37E+08	-290960	-4863	5952
	DelayGBM	5.12E+08	-233211	-4196	2515
.	DREAMPlace	0.97	1.34	1.29	0.82
	DREAMPlace4.0	1.01	1.24	1.14	1.11
Avg	MLCAD'22	1.02	1.26	1.12	2.56
	DAC'22	1.05	1.22	1.12	2.59
	DelayGBM	1.00	1.00	1.00	1.00

The TNS and WNS of placers are obtained according to the Elmore-based evaluation [27].

Just Accepted

7.4 Discussion on Derivable Timing Model

Recent studies [20, 38] have also explored derivable AI models such as GNNs and Transformers for timing prediction. To evaluate our approach with other derivable models, we give the prediction correlation analysis on pin-to-pin delay and path arrival time as shown in Table 8.

For the GNN-based timing prediction model [20], a direct comparison is not possible since it requires a specific library format as input for GNN embedding and has not been released for the 28 nm technology node used in our experiments. To address this, we re-implemented the GNN model, preserving its key components: net embedding and delay propagation, while adapting it to our experimental environment. Our implementation employs the same feature set as "delayGBM", trained with the Adam optimizer, mixed-precision training, and MSE loss function.

As shown in Table 8, compared to "delayGBM", which achieves a pin-to-pin delay correlation of 0.96, the correlation of the GNN model is 0.94. The average correlation of path arrival time is also lower than delayGBM, especially for larger circuits. In addition, circuit graph construction for the GNN model introduces significant complexity. The GNN inference time is about 20∼100x longer, averaging more than 90x longer than "delayGBM". While the primary strength of GNNs lies in their ability to automatically learn graph features, this advantage may be less pronounced when highly correlated features are already explicitly provided, suggesting that the potential of GNNs in our context requires further investigation. Since the GNN model is significantly more complex than "delayGBM", we believe that a more refined, GNN-specific embedding strategy is required during placement, which we leave for future work.

For the Transformer model, work [38] utilizes its sequence modeling capability to predict path timing. However, timing paths are not simple linear chains but instead form directed acyclic graphs with branching structures. As a result, the method in [38] cannot be directly applied in our setting. Inspired by this work, we construct a Transformer-based model for pin-to-pin timing prediction. This model adopts the same features and loss function

Table 8. Derivable Model Comparison on Timing Correlation

	GNN		Transformer		DelayGBM	
Circuit	Delay	Path AT	Delay	Path AT	Delay	Path AT
b11	0.91	0.94	0.85	0.84	0.92	0.90
b12	0.91	0.73	0.86	0.71	0.93	0.78
b14	0.95	0.98	0.87	0.94	0.95	0.96
b17	0.95	0.97	0.94	0.95	0.95	0.97
b22	0.95	0.98	0.92	0.96	0.96	0.98
CV32	0.96	1.00	0.94	1.00	0.97	1.00
CVA6	0.96	0.99	0.96	0.98	0.98	0.99
C910	0.93	0.94	0.97	0.97	0.98	0.98
Avg.	0.94	0.94	0.91	0.92	0.96	0.95

as our proposed "delayGBM", while applying binning and encoding during preprocessing and employing a multi-head attention mechanism for prediction. As shown in Table 8, the Transformer model achieves an average delay correlation of 0.91, which is lower than the 0.96 achieved by "delayGBM".

Theoretically, the Transformer model can be directly integrated into the placement process. However, this would require developing methods to derive gradients from the Transformer and use them to guide timing optimization, which is beyond the scope of this work. Therefore, we instead adopt the same net-weighting integration method used in the "DAC'22" and "MLCAD'22" studies, and the runtime overhead of calling Transformer model is not accounted for a fair comparison. Using the Transformer for net weighting yields an average HPWL nearly identical to that of "delayGBM". However, it produces worse timing results on average, with TNS and WNS being 29% and 20% higher, respectively.

7.5 Discussion on Model Application

In recent years, with the growing research on timing prediction models, increasing attention has been paid to their integration into design flows. During timing-driven placement, timing optimization methods can be built upon classical placement algorithms, where nets with timing violations receive special treatment. By adjusting net weights, or path constraints, timing convergence can be achieved through improved delay. This work proposes a derivable timing prediction model and use its derivablility to guide net weighting. Furthermore, for path constraints methods like [19], the derivable model can, in principle, serve as a substitute for Elmore delay in constructing constraint functions.

In addition, prior works [4, 15] have explored the use of timing prediction models to guide commercial EDA tool optimization via timing annotation commands. Unlike the timing-driven optimizations discussed above during placement, these approaches allow for more aggressive and flexible design modifications, including buffer insertion, gate sizing, and logic restructuring.

It should be noted that the effectiveness of integrating prediction models through path constraints or tool commands requires further investigation. In principle, the predictions from our model or other AI-based models could be used with tool commands to enable more accurate timing analysis, allowing the tool to apply more effective optimization strategies. Specifically, open questions include whether to adopt refined path-margin annotation ratios to adjust predicted slack [4], apply path grouping [15], and other related strategies. These directions will be studied in future work.

8 Conclusion

In this paper, we introduce an innovative timing prediction model that can be transformed into a derivable function, serving as a replacement for the conventional Elmore method. To evaluate the effectiveness of our

Just Accepted

timing model, we integrate it into the open-source placer DREAMPlace. We also utilize a pin-to-pin weighting approach based on derivable timing model is given for timing optimization. Experimental results show that our derivable timing prediction model can significantly reduce the max and mean timing errors compared to the Elmore delay, and exhibits equivalent to or even better than previous non-derivable models. The timing performance after placement optimization is better than the result using Elmore delay or non-derivable model, i.,e., 12%∼24% smaller TNS and WNS with wirelength decreases by 1%∼5% on average.

Acknowledgments

This work is supported by the National Natural Science Foundation of China, under grant 92473115.

References

[1] [n. d.]. CV32E40P:PULP-Platform 32-bit 4-stage RISC-V CPU Core. https://github.com/openhwgroup/cv32e40p.
[2] [n. d.]. CVA6 RISC-V CPU. https://github.com/openhwgroup/cva6.
[3] [n. d.]. T-HEAD XuanTie C910 RISC-V. https://github.com/XUANTIE-RV/openc910.
[4] Jaehoon Ahn, Kyungjoon Chang, Kyu-Myung Choi, Taewhan Kim, and Heechun Park. 2024. DTOC-P: deep-learning-driven timing optimization using commercial EDA tool with practicality enhancement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 8 (2024), 2493–2506.
[5] C. J. Alpert, A. Devgan, and C. Kashyap. 2001. A Two Moments RC Delay Metric for Performance Optimization. In *International Conference on Computer-Aided Design*. 69–74.
[6] E. C. Barboza, N. Shukla, Y. Chen, and J. Hu. 2019. Machine Learning-Based Pre-Routing Timing Prediction with Reduced Pessimism. In *56th ACM/IEEE Design Automation Conference*. 1–6.
[7] A. Bock, S. Held, N. Kammerling, and U. Schorr. 2015. Local search algorithms for timing-driven placement under arbitrary delay models. In *DAC*. 1–6.
[8] H Chang, E. Shragowitz, J. Liu, and et al. 2002. Net criticality revisited: An effective method to improve timing in physical design. In *ISPD*. 155–160.
[9] Hsien-Han Cheng, Iris Hui-Ru Jiang, and Oscar Ou. 2020. Fast and accurate wire timing estimation on tree and non-tree net structures. In *DAC*. IEEE, 1–6.
[10] V. A. Chhabria, W. Jiang, A. B. Kahng, and S. S. Sapatnekar. 2022. From Global Route to Detailed Route: ML for Fast and Accurate Wire Parasitics and Timing Prediction. In *ACM/IEEE 4th Workshop on Machine Learning for CAD (MLCAD)*. 7–14.
[11] A. Chowdhary, K. Rajagopal, S. Venkatesan, and et al. 2005. How accurately can we model timing in a placement engine?. In *DAC*. 801–806.
[12] C. Chu. 2004. FLUTE: fast lookup table based wirelength estimation technique. In *ICCAD*. 696–701.
[13] S. Davidson. 1999. Characteristics of the ITC'99 Benchmark Circuits. In *IEEE International Test Synthesis Workshop*.
[14] William C Elmore. 1948. The transient response of damped linear networks with particular regard to wideband amplifiers. In *Journal of Applied Physics*. 55–63.
[15] Wenji Fang, Shang Liu, Hongce Zhang, and Zhiyao Xie. 2024. Annotating slack directly on your verilog: Fine-grained rtl timing evaluation for early optimization. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
[16] J. Feng, Y. Xu, Y. Jiang, and Z. Zhou. 2020. Soft gradient boosting machine. *arXiv* (2020).
[17] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting deep learning models for tabular data. *Advances in Neural Information Processing Systems* 34 (2021), 18932–18943.
[18] G. Guo, T. Huang, C. Lin, and M. Wong. 2020. An efficient critical path generation algorithm considering extensive path constraints. In *DAC*. IEEE, 1–6.
[19] Z. Guo and Y. Lin. 2022. Differentiable-timing-driven global placement. In *DAC*. 1315–1320.
[20] Z. Guo, M. Liu, J. Gu, and et al. 2022. A timing engine inspired graph neural network model for pre-routing slack prediction. In *DAC*. 1207–1212.
[21] S. Han, A. B. Kahng, S. Nath, and et al. 2014. A deep learning methodology to proliferate golden signoff timing. In *2014 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1–6.
[22] X. He, Z. Fu, Y. Wang, and et al. 2022. Accurate timing prediction at placement stage with look-ahead RC network. In *DAC*. 1213–1218.
[23] Tsung-Wei Huang and Martin D. F. Wong. 2015. OpenTimer: A high-performance timing analysis tool. In *IEEE/ACM International Conference on Computer-Aided Design*. 895–902.
[24] A. B. Kahng, S. Kang, H. Lee, and et al. 2013. Learning-based approximation of interconnect delay and slew in signoff timing tools. In *SLIP*. IEEE, 1–8.

Just Accepted

[25] A. B. Kahng, M. Luo, and S. Nath. 2015. SI for free: machine learning of interconnect coupling delay and transition effects. In *2015 ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP)*. IEEE, 1–8.
[26] A. B. Kahng, U. Mallappa, and L. Saul. 2018. Using machine learning to predict path-based slack from graph-based timing analysis. In *ICCD*. IEEE, 603–612.
[27] Myung-Chul Kim and et al. 2015. ICCAD-2015 CAD contest in incremental timing-driven placement and benchmark suite. In *ICCAD*. 921–926.
[28] Tai-Cheng Lee and Yih-Lang Li. 2019. Incremental timing-driven placement with approximated signoff wire delay and regression-based cell delay. *TVLSI* 27, 10 (2019), 2434–2446.
[29] P. Liao, D. Guo, Z. Guo, S. Liu, Y. Lin, and B. Yu. 2023. DREAMPlace 4.0: Timing-driven Placement with Momentum-based Net Weighting and Lagrangian-based Refinement. *TCAD* (2023).
[30] Y. Lin, S. Dhar, W. Li, H. Ren, and et al. 2019. Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement. In *DAC*. 1–6.
[31] D. S. Lopera and W. Ecker. 2022. Applying GNNs to Timing Estimation at RTL. In *ICCAD*. 1–8.
[32] T. Luo, D. Newmark, and D. Z. Pan. 2006. A new LP based incremental timing driven placement for high performance designs. In *DAC*. 1115–1120.
[33] Ravi Nair, C Leonard Berman, Peter S Hauge, and Ellen J Yoffa. 1989. Generation of performance constraints for layout. *TCAD* 8, 8 (1989), 860–874.
[34] M. Orshansky and K. Keutzer. 2002. A General Probabilistic Framework for Worst Case Timing Analysis. In *Proceedings of the 39th Annual Design Automation Conference*. 556–561.
[35] H. Ren, D. Z. Pan, and D. S. Kung. 2005. Sensitivity guided net weighting for placement-driven synthesis. *TCAD* 24, 5, 711–721.
[36] Q. Song, X. Cheng, and P. Cao. 2023. Critical Paths Prediction under Multiple Corners Based on BiLSTM Network. In *DAC*. IEEE, 1–6.
[37] Z. Xie, R. Liang, X. Xu, J. Hu, and et al. 2022. Preplacement net length and timing estimation by customized graph neural network. *TCAD* 41, 11 (2022), 4667–4680.
[38] Tai Yang, Guoqing He, and Peng Cao. 2022. Pre-routing path delay estimation based on transformer and residual framework. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 184–189.

Received 9 March 2025; revised 14 October 2025; accepted 23 November 2025

Just Accepted